PROOF OF CONCEPT PLAN

CastIron Media — Phase 1 Proof of Concept

This document defines a time-boxed proof of concept that validates the four highest-risk, most novel pieces of the recommended Phase 1 architecture in a sandbox AWS account before any production build begins, and prices that proof of concept on its own, separately from the Phase 1 production run-cost estimate.

1. Purpose and Relationship to the Technical Approach

The Phase 1 Technical Approach document (companion to this one) sets out the production architecture and its steady-state run cost. Before committing to that production build, this proof of concept proves the architecture's riskiest and least-precedented mechanisms actually work as designed, at a small scale, in a real (but sandboxed) AWS environment — not in a slide. Where the production document estimates performance and cost from vendor rate cards, this document is designed to produce measured numbers instead of assumed ones.

> **What this POC is not**
>
> It is not a scaled-down production deployment, not a pilot customers can see, and not a substitute for the shadow-run parity process in the Technical Approach document (Section 2, Section 7.1 of that document). It is disposable: the sandbox account and everything in it can be torn down at the end without any impact on production.

2. What This POC Proves

#	Objective	Why it is the highest-risk item to prove first
1	Kinesis EFO → Lakeflow Declarative Pipeline streaming end-to-end, bronze through gold	The production latency figures (1–2 min to silver, 5–10 min to gold) are vendor-rate-card estimates, not measurements. Kinesis shard sizing is also still unconfirmed (Open Items, Technical Approach §9).
2	Identity resolution join at representative scale	The 8M-probe-against-3.2B-identifier join is join-bound, not ingest-bound — its cost and runtime depend on data skew and cluster shape that only a real join can reveal.
3	The Snowflake clean-room pattern, end to end	This is the hard requirement from the brief and the least precedented integration in the architecture: CastIron's own Snowflake provider account, a same-region catalog integration, a read-only Iceberg table, and a linked offering into a second (test) Snowflake account's Collaboration Data Clean Room.
4	Terraform layered delivery through GitHub Actions	Proves the L0–L5 module layering (Technical Approach §7) actually plans and applies cleanly end-to-end, and that the pipeline's approval gates work, before it is relied on for production.

3. Scope

3.1 In scope

- A single sandbox AWS account and a single Databricks workspace — no dev/staging/production split.
- A synthetic event generator producing a representative but reduced volume (target: 2–3% of production throughput, ~150–200 events/s) — enough to exercise the pipeline's logic and autoscaling behavior without the cost of full-scale load.
- A sampled identifier table (target: 10–20M rows, not the full 3.2B) and a proportionally sampled daily probe file, used to measure join behavior and extrapolate full-scale runtime and cost.
- A second, CastIron-owned Snowflake account standing in for the partner's account, used only to prove the clean-room mechanics — registering an offering, linking it, and running an approved aggregate query against it.
- Terraform for every layer in Section 4, deployed through a GitHub Actions pipeline with a manual approval gate before apply.

3.2 Out of scope

- Any connection to the real Omnicom Snowflake account, or any real partner data.
- Full-volume load testing at 562.7M events/day — throughput conclusions are extrapolated from the reduced-volume run, not measured directly at full scale.
- The licensed third-party device graph — the licensor has not been selected (Technical Approach §9), so identity resolution is proved against a synthetic identifier table, not a real licensed graph.
- Production cutover, historical backfill, and the shadow-run parity process against Redshift.
- Exact measurement business logic (reach, frequency, equivalization) — the POC proves the mechanism (joins, aggregation, Auto Loader validation ingest) runs correctly, not the final certified numbers.

4. POC Architecture

The POC deploys the same component types as the Phase 1 production architecture — Kinesis, Lakeflow, Unity Catalog, Databricks SQL, the Snowflake provider pattern — at reduced scale and in a single environment, so that what is measured here transfers directly to the production sizing in the Technical Approach document. See that document's Figure 1 for the full target-state diagram; the POC stands up the same shape with a synthetic source in place of the live ad server and a test Snowflake account in place of Omnicom's.

Component	Production (Technical Approach)	POC (this document)
Event source	Live ad server, ~6,500 events/s	Synthetic generator, ~150–200 events/s
Kinesis	Shard count to be confirmed for ~25K/s bursts	Fixed at 4 shards — enough to prove the EFO consumer pattern
Identity graph	Licensed, 3.2B rows, resident or hosted	Synthetic sample, 10–20M rows, resident
Environments	Dev, staging, production (separate Terraform state per layer)	Single sandbox environment
Snowflake partner side	Omnicom's own account, same region	A second CastIron-owned test account standing in for the partner
Duration	Ongoing	Time-boxed, 3 weeks

5. Success Criteria and Exit

Objective	Pass criterion
Streaming end-to-end	Synthetic events observable in the gold table within the latency band measured, with zero data loss across a forced worker restart
Identity join	Join completes without manual tuning; runtime and cluster cost measured and extrapolated to the full 3.2B-row table
Clean room	An aggregate query run from the test partner account against the linked offering returns correct results, and a direct row-level query against the source table from the partner side is confirmed to fail
Terraform/CI-CD	A full L0–L5 apply succeeds from a clean account via the GitHub Actions pipeline, including the manual approval gate, and a subsequent plan shows no drift

The POC exits either by all four objectives passing — at which point the measured numbers replace the vendor-rate-card estimates in the Technical Approach document before production build starts — or by a documented failure on any objective, in which case the architecture for that component is revisited before production build starts. No production build begins on an objective that has not passed.

6. POC Cost Estimate

This is priced independently from the Phase 1 production run cost (Technical Approach §8, ~USD 118,000/year steady state). The POC runs for 3 weeks in a single, smaller, non-redundant environment, so the basis of estimate is materially different — shown explicitly per line so it can be checked.

Workload	Resource and sizing	Usage basis (3-week POC)	Databricks (USD)	AWS (USD)	Total (USD)
Event ingest	Kinesis EFO consumer, 4 shards	21 days, 24x7	-	70	70
Streaming pipeline	Lakeflow Declarative Pipeline (Core), classic; driver + 1 x r6gd.xlarge	21 days, 24x7; ~2 DBU/h	220	165	385
Identity resolution test	Jobs compute; driver + 2 x r6gd.xlarge (spot); 10–20M-row sample	5 test runs, ~1 h each	25	10	35
Clean-room contribution	Jobs compute; driver + 1 x r6gd.xlarge	10 test runs, ~30 min each	10	5	15
Snowflake — CastIron provider account	X-Small warehouse, catalog integration	21 days, on-demand use only	-	-	90
Snowflake — test partner account	X-Small warehouse, Collaboration Data Clean Room	21 days, on-demand use only	-	-	90
Lakehouse storage	S3 Standard, synthetic + sampled data	~200 GB for 21 days	-	10	10
Network and security	Single-AZ NAT, KMS, CloudWatch	21 days	-	60	60

Terraform/CI-CD runs	GitHub Actions compute (plan/apply across L0–L5, ~15 runs)	Included in GitHub plan or ~15 runner-hours	-	15	15
Total (3-week POC)			**255**	**335**	**~770**

Estimated total: approximately USD 770 for the 3-week proof of concept (Snowflake figures are planning estimates pending the account's actual credit rate; all other figures follow the same unit rates as Technical Approach §9). This is separate from, and much smaller than, the USD 118,000/year Phase 1 production run cost — the POC proves the architecture, it does not run it at scale.